Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 12:
Costrutti Avanzati con i Semafori

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'Hands-On 12 espande l'orizzonte delle problematiche legate alla concorrenza esplorando due classici pattern architetturali della sincronizzazione multi-thread: 1. **Meccanismo di Barriera**: Realizzazione di un sistema capace di costringere un insieme di N thread ad attendersi reciprocamente in un determinato checkpoint per poter riprendere l'esecuzione in modo perfettamente simultaneo. 2. **Produttore-Consumatore con Buffer Limitato**: Implementazione del celebre costrutto in cui un processo produce dati da inserire in una coda a capienza limitata, e un altro consuma i suddetti elementi. Il sistema non deve presentare né overflow (scrittura su buffer pieno) né underflow (lettura su buffer vuoto).

2 Stato dell'Arte

Il controllo della sincronizzazione passa attraverso i semafori generalizzati.

- **Barriere (Barrier Synchronization)**: Sono punti di blocco in cui un thread non può procedere a meno che tutti gli altri compagni abbiano anch'essi raggiunto lo stesso blocco logico. Sono ampiamente impiegate in logiche multi-step e nei branch di algoritmi genetici paralleli o reti neurali.
- **Produttore-Consumatore**: È un *design pattern* essenziale in scenari di buffering di I/O, streaming audio-video e pipeline di elaborazione asincrona. I **semafori** fungono da mediatori, mantenendo implicitamente lo stato di svuotamento/riempimento del buffer condiviso, sostituendosi a complessi controlli espliciti della dimensione tramite `if` ed estenuanti cicli d'attesa (spinlocks o active-wait).

3 Metodologia ed Implementazione

3.1 Implementazione Barriera (Barrier)

L'approccio prevede l'utilizzo di un contatore tenuto al sicuro da un `sem_t mutex` e di un semaforo di arresto `barrier_sem`. I primi $(N-1)$ thread che effettuano gli accessi andranno tutti rigorosamente posteggiati ed addormentati chiamando `sem_wait(&barrier_sem)`. Solo l'arrivo dell'ultimo thread sancirà la fine dell'attesa in quanto quest'ultimo innescherà una cascata di `sem_post(&barrier_sem)` pari al numero di thread fermi $(N-1)$.

3.2 Implementazione Produttore-Consumatore

Il buffer circolare finito è gestito da tre entità di blocco.

Snippet Esercizio 2: Tre Semafori Coordinati

```
// Sincronizzazione:
sem_init(&empty_slots, 0, BUFFER_SIZE);  // Attesta quanti spazi sono vuoti
sem_init(&filled_slots, 0, 0);           // Attesta quanti spazi sono pieni
sem_init(&mutex, 0, 1);                  // Regolamenta l'esclusione sul buffer

// Lato Produttore:
sem_wait(&empty_slots);  // Preleva uno slot "vuoto"
sem_wait(&mutex);        // Entra
buffer[in] = item;       // PRODUCE
sem_post(&mutex);        // Esce
sem_post(&filled_slots); // Regala un token "pieno" al consumatore

// Lato Consumatore:
sem_wait(&filled_slots); // Preleva uno slot "pieno"
sem_wait(&mutex);        // Entra
int item = buffer[out];  // CONSUMA
sem_post(&mutex);        // Esce
sem_post(&empty_slots);  // Converte il "pieno" appena letto in "vuoto"
```

3.3 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

Build Run

```
$ make
$ ./programma
```

4 Risultati

L'esecuzione del codice ha fornito log testuali perfetti di dimostrazione del comportamento atteso.

4.1 Output Barriera

Output di Esecuzione - Memoria Condivisa

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho12$
   ./ex1_barrier
Thread 1: Arrivato alla barriera (2/5).
Thread 2: Arrivato alla barriera (3/5).
Thread 3: Arrivato alla barriera (4/5).
Thread 4: Arrivato alla barriera (5/5).
Thread 4: E' l'ultimo! Sblocco la barriera per tutti.
Thread 4: Ha superato la barriera e sta eseguendo il suo
   compito!
Thread 1: Ha superato la barriera e sta eseguendo il suo
   compito!
...
```

Si osserva come i thread si incodino rimanendo freezati fino a quando la stampa *"E' l'ultimo!"* del Thread 4 fa scattare esecuzioni libere contemporanee per tutti.

4.2 Output Produttore-Consumatore

Sincronizzazione tramite Semafori

```
Produttore 1: [PRODOTTO] -> 109 in pos 4
Consumatore 1: [CONSUMATO] <- 105 in pos 0
...
Consumatore 1: [CONSUMATO] <- 113 in pos 3
Consumatore 1: [CONSUMATO] <- 114 in pos 4
Esecuzione Completata. Buffer Gestito con Successo.
```

L'esito dimostra un array circolare perfettamente equilibrato in cui il processo più rapido (spesso il produttore) per forza di cose attende il termine del buffer `109 in pos 4` bloccandosi finché il consumatore non libera le caselle precedenti permettendogli l'ingresso `110 in pos 0`.

5 Conclusioni e Sviluppi Futuri

Gli strumenti elaborati sanciscono come il concetto di semaforo permetta la conversione di complessi ragionamenti concorrenziali in un semplice ed efficiente "passaggio di token", capace di prevenire l'overhead della CPU senza sacrificare le performance.
Sviluppi futuri:

- **POSIX Barriers**: Nei contesti produzione reale con standard attuali, andrebbe utilizzato `pthread_barrier_t` (meccanismo nativo POSIX) invece della complessa re-implementazione custom a semafori.

- **Cond Variables (CV)**: Ottimizzazione del produttore consumatore utilizzando le Variabili Condizionali insieme a un solo mutex anziché usare molteplici semafori a conteggio, approccio standard nei monitor o in programmazione ad alto livello.